

Project 3: Errors in Numerical Integration of ODEs

In class we talked about various methods for the numerical integration of ordinary differential equations (ODEs), and took a very careful look at their error properties. Here you will verify statements about error sizes in these integrators, using a very simple ODE that is easy to solve analytically.

Consider one of the simplest linear ODEs in existence:

$$\frac{dN}{dt} = -\mu N \quad (1)$$

with initial condition $N(t = 0) = N_0$.

We will start by taking a different approach to error in the Euler method. What if we treated the results of a numerical integration of this ODE as “data,” and used it to compute μ , the decay constant? How would that behave with stepsize?

- (a) Obtain an analytical solution to Eqn. 1.
- (b) Suppose you have integrated the equation from $t = 0$ to $t = T$, some later time. How would you use the results of that integration to calculate the value of μ ? (*Hint:* There are many possible answers here, but pick the easiest one! You should be able to do it using only two points — $N(T)$ and $N(0)$.)
- (c) Write down the Euler step for solving this ODE — specifically, what should $N_{t+\Delta t}$ look like using the Euler method?
- (d) Now write a Python script/function that integrates Eqn. 1, using the Euler method, for $\mu = 0.8$, $N_0 = 0.1$, a total time of $T = 4$, and a number of steps $M = 25$. (So your stepsize Δt , which I called h in class, will be equal to $T/M = 0.16$.) Use the output of your script to calculate μ , using the formula you derived in part (b). Do you get back the μ you put in?

You should find that your estimate of μ doesn't match what you put in! This is a reflection of the accuracy issues with the Euler method; μ is wrong because our stepsize is causing our solution to be inaccurate. For an ODE this simple, we can do a couple more calculations to see this explicitly.

- (e) Go back to your difference equation; you should be able to obtain an analytical expression for $N_T = N_{M\Delta t}$. (*Hint:* Start by finding $N_{\Delta t}$, the value after a single step, starting at $t = 0$. Then do $N_{2\Delta t}$, which uses the value you just found for $N_{\Delta t}$. Continue until you figure out the pattern.)

Notice that $N_{M\Delta t}$ doesn't look like the analytical solution you found in part (a).

- (f) Using the definition of e^x as a limit

$$e^x = \lim_{n \rightarrow \infty} \left(1 + \frac{x}{n}\right)^n \quad (2)$$

show that your difference equation solution converges to the true solution as $\Delta t \rightarrow 0$ — or equivalently $M \rightarrow \infty$ — for a fixed final time T . (*Hint:* You may need to play around with some substitutions using $T = M\Delta t$ to get the limit expression in the form you need.)

So now you have found that you need to take *extremely* small time steps in order for the Euler method to produce an acceptable answer (as I told you was the case in class). What if we shrink the stepsize a bit?

- (g) Use your Python script to take $M = 250$ steps and measure μ as you did in part (d). Is it closer to the number you expect?
- (h) Now let's measure the error in N_t as a function of stepsize. Calculate the absolute error ε_G using your Euler integration (out to N_T) and the exact solution for $N(T)$ that you found in (a) for a few (ten at the least!) values of M in the range $M = 25$ to $M = 25000$ (corresponding to smaller and smaller stepsizes). Plot your calculated absolute errors as a function of M (or $\Delta t = h$ if you prefer) in a double logarithmic plot (in `matplotlib` you accomplish this with the function `loglog`). What will make the plot look the nicest is if you space your M values logarithmically; the function `kb_logspace` will do this for you. Based on my lectures, you should know how ε_G scales with M . Does it look like it follows the theory?

Ok, but we know we can do better. In the next few parts of the problem, rather than measuring the absolute error, you will measure the percent error for a few different integration schemes. If you have a true value v_t and an estimated value v_e , the percent error between your estimated and true values is defined as

$$\varepsilon_p = 100 \frac{|v_t - v_e|}{|v_t|} \quad (3)$$

and in what follows, your “true” value is what your analytical solution gives you and the “estimated” value is what the numerical integration produces.

- (i) Write another Python function that uses a second-order Runge-Kutta scheme to integrate Eqn. 1.
- (j) Now integrate Eqn. 1 with both your RK and Euler integrators, out to $T = 4$, using a stepsize (Δt) of 0.25. Measure the percent error in $N(T)$ for both methods. What do you find?

You should have seen a substantial difference in percent error between the two methods. Finally, we can try out an implicit integrator.

- (k) Write a third and final Python function that integrates Eqn. 1 using the implicit Euler method I discussed in class. For a stepsize of 0.25, what's the percent error for the implicit Euler method?
- (l) Finally, as I remarked in class, one of the main benefits of implicit methods isn't error rate, it is stability at large step sizes. Integrate (one last time!) using both regular and implicit Euler with a $\Delta t = 4$, i. e. attempt to take a single step to the end. What are your percent errors?

And there you have it. Only use explicit Euler if you absolutely have to. Higher-order Runge-Kutta methods are a best “first try” integrator in most cases. If you are having stability problems (getting wacko results), switching to an implicit (or other stiff) method is your best bet.